

1. Data Cleaning Strategy:

i. Handling Missing Values

Books Dataset:

- Drop rows missing essential info like Book-Title, Book-Author, or Publisher.
- Convert Year-Of-Publication to numeric.
- Converted invalid Year-Of-Publication entries (e.g., text) to NaN.
- Impute Year-Of-Publication using:
 - a. Replace with median year of the dataset if ISBN exists multiple times with valid years.
 - b. Mode of year column, if ISBN is unique.

Users Dataset:

- Fill missing Location with "Unknown".
- Fill missing Age with median/Mode
- drop if Age outside valid range (e.g., 5–100).

Ratings Dataset:

- Drop rows with missing User-ID or ISBN .

ii. Removing Duplicates

Books dataset :

- Keep only one row per ISBN if metadata is identical.

Users dataset:

- Ensure User-ID is unique.
- Remove exact duplicate user records.

Ratings dataset:

- Remove rows with the same User-ID, ISBN, and Book-Rating.

iii. Standardizing Data Formats

a. Text Fields (titles, authors, publishers, Location):

- Strip whitespaces.
- Convert all text fields (Book-Title, Book-Author, Publisher, Location) to lowercase.
- Normalize publisher/author spellings.
- Clean special characters (_ , * , -) in Location.

b. Date Fields:

- Convert Year-Of-Publication to integer and apply standardization.
- Remove or fix non-year values (e.g., text in place of year).

c. User Age:

- Set valid age range (e.g., keep only $5 \leq \text{Age} \leq 100$).

iv. Fixing Inconsistencies

- Replace incorrect Year-Of-Publication (e.g., < 1800 or > 2025) with Median or most frequent value for that ISBN.
- Remove invalid Publisher values (e.g., numeric-only names).
- Normalize Book-Rating: removing implicit ratings (i.e., 0) if not useful for modeling.

v. Outlier Detection

- Draw boxplot to detect outliers

a. Age (Users)

- Apply the Interquartile Range (IQR) method:
- Removed users with ages beyond $1.5 \times \text{IQR}$ from Q1 and Q3.
- Helps to eliminate improbable outliers like 0, 200, etc.

b. Year-Of-Publication (Books)

- Apply IQR filtering after removing invalid years (e.g., 0, 9999).
- Helped in identifying extremely old or wrongly entered publication years.

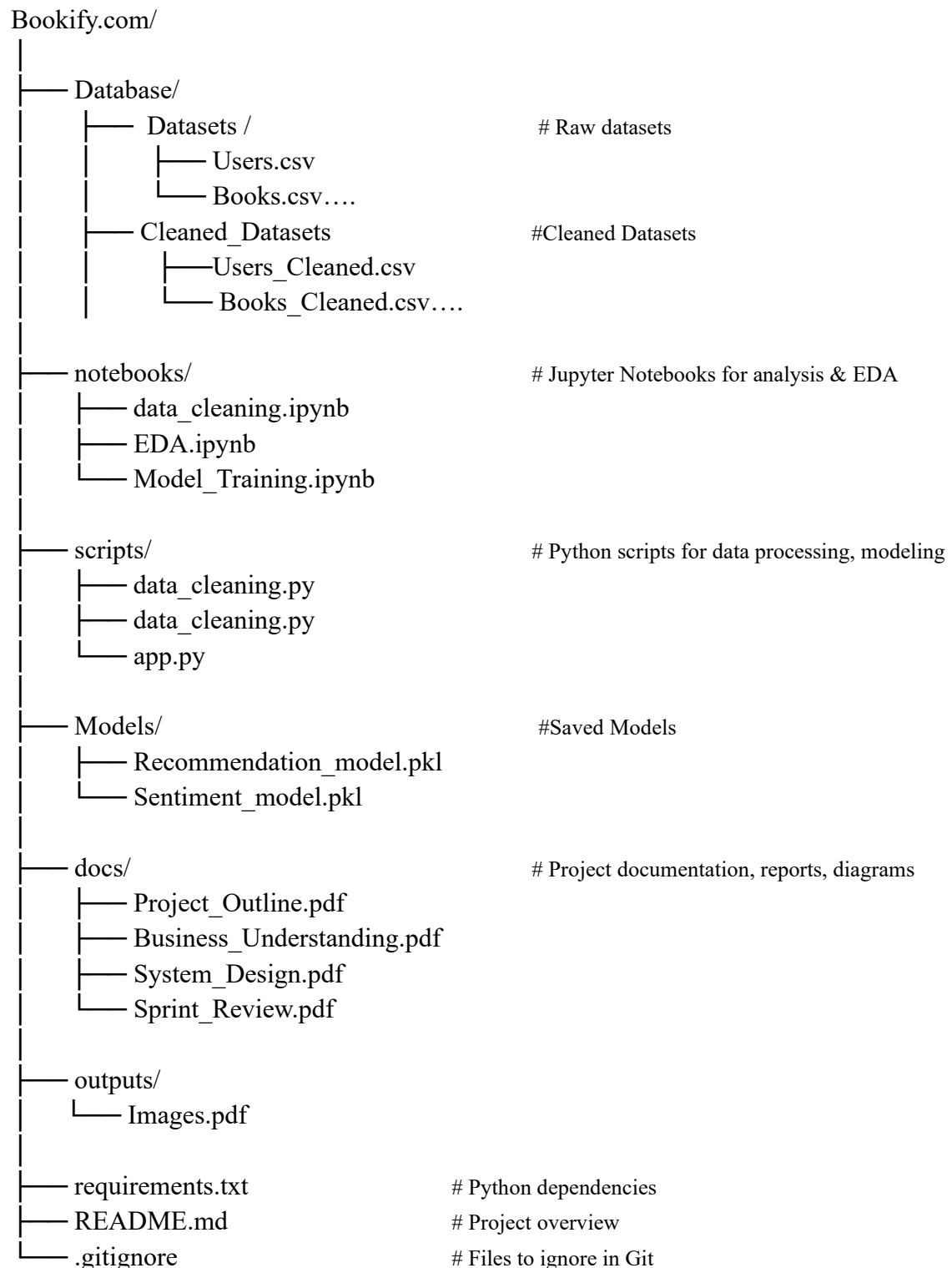
vi. General Cleaning Practices

- Use `.dropna()` for rows with critical null values.
- Use `.fillna()` for columns where imputation makes sense (e.g., age).

- Apply `.drop_duplicates()` to clean repeated data entries.
- Standardized ISBN as string format; validated length (10 or 13 characters).

2. Setting Up a Git Repository:

Repository Structure:



3. Data Validation Approach

i. Accuracy of Values (Correct Format / No Corruption)

Checks Performed:

- Ensure columns are in expected formats (e.g., rating is numeric, user_id and book_id are strings).
- Detect unusual or corrupted text.

```
In [4]: 1 # Books Dataset: Check ISBN format (ensure 10 or 13 digits)
2 books_invalid_isbn = cl_book_df[~cl_book_df['book_id'].str.match(r'^\w{10}(\w{3})?$', re.I)]
3 if not books_invalid_isbn.empty:
4     print(f"Books with invalid Book ID format:\n{books_invalid_isbn[['book_id']]}\n")
5 else:
6     print("All books have valid Book ID formats.\n")
```

```
Books with invalid Book ID format:
      book_id
111808  0486404242\t
171206  3518365479<90
251424  3442248027 3
251649  0385722206 0
```

```
In [5]: 1 # Users Dataset: Check valid age range (e.g., 5-100)
2
3 invalid_users_age = cl_user_df[~cl_user_df['age'].between(5, 100)]
4 if not invalid_users_age.empty:
5     print(f"Users with invalid age:\n{invalid_users_age[['user_id', 'age']]}\n")
6 else:
7     print("All users have valid ages within the 5-100 range.\n")
8
```

All users have valid ages within the 5-100 range.

```
In [6]: 1 # Ratings Dataset: Check for valid ratings range (1-10)
2
3 invalid_ratings = cl_rating_df[~cl_rating_df['ratings'].between(1, 10)]
4 if not invalid_ratings.empty:
5     print(f"Ratings outside valid range (1-10):\n{invalid_ratings[['user_id', 'book_id', 'ratings']]}\n")
6 else:
7     print("All ratings are within the valid range (1-10).\n")
```

All ratings are within the valid range (1-10).

```
In [7]: 1 # Reviews Dataset: Check for corrupted text or empty reviews
2 empty_reviews = cl_pre_df[cl_pre_df['review_text'].isna() | (cl_pre_df['review_text'].str.strip() == '')]
3 if not empty_reviews.empty:
4     print(f"Empty or corrupted reviews found:\n{empty_reviews[['user_id', 'book_id', 'review_text']]}\n")
5 else:
6     print("All reviews are non-empty and properly formatted.\n")
```

All reviews are non-empty and properly formatted.

ii. Consistency Across Records (Dangling References)

- Each rating and review must link to valid user_id and book_id from users and books.

```
In [8]: 1 # Ensure no dangling references (i.e., user_id and book_id in ratings/reviews exist in users/books)
2 missing_users_in_ratings = cl_rating_df[~cl_rating_df['user_id'].isin(cl_user_df['user_id'])]
3 missing_books_in_ratings = cl_rating_df[~cl_rating_df['book_id'].isin(cl_book_df['book_id'])]
4 missing_users_in_reviews = cl_pre_df[~cl_pre_df['user_id'].isin(cl_user_df['user_id'])]
5 missing_books_in_reviews = cl_pre_df[~cl_pre_df['book_id'].isin(cl_book_df['book_id'])]
6
7 if not missing_users_in_ratings.empty:
8     print(f"Ratings with missing users:\n{missing_users_in_ratings[['user_id', 'book_id']]}")
9 else:
10    print("All ratings reference valid users.\n")
11
12 if not missing_books_in_ratings.empty:
13    print(f"Ratings with missing books:\n{missing_books_in_ratings[['user_id', 'book_id']]}")
14 else:
15    print("All ratings reference valid books.\n")
16
17 if not missing_users_in_reviews.empty:
18    print(f"Reviews with missing users:\n{missing_users_in_reviews[['user_id', 'book_id']]}")
19 else:
20    print("All reviews reference valid users.\n")
21
22 if not missing_books_in_reviews.empty:
23    print(f"Reviews with missing books:\n{missing_books_in_reviews[['user_id', 'book_id']]}")
24 else:
25    print("All reviews reference valid books.\n")
```

Ratings with missing users:

	user_id	book_id
1248	277909	0896592413
1809	278302	2070495027
3532	932	4491217967
4089	1461	0816614024
4463	1850	0207185905
...	...	...
428523	272989	0061096105
428524	272989	0061097101
428525	272989	034543479X
428526	272989	0451204891
432781	276074	0865470014

[2177 rows x 2 columns]

Ratings with missing books:

	user_id	book_id
3	276736	3257224281
4	276737	0600570967
6	276745	342310538
16	276760	8440682697
19	276762	3453213025
...	...	...
433518	276670	15655122046
433522	276670	55305663
433527	276679	8804413530
433597	276680	0679781587
433670	276723	05162443314

[49829 rows x 2 columns]

Reviews with missing users:

	user_id	book_id
0	A3HHXRELK8BHQG	B0033UV8HI
1	A2RGNZ0TRF578I	B002HJV4DE
2	A3S0H2HV6U1I7F	B002ZG96I4
3	AC40QW3GZ919J	B002QHW0EU
4	A3C9V987IQHOQD	B001A06VJ8
...	...	...
77141	Carol McCoy	1492646318
77142	Chelscey	1501161938
77143	bayrider	385534264
77144	Amazon Customer	B0B6XFT4RH
77145	Jeanette F Reeder	1728213746

[77146 rows x 2 columns]

Reviews with missing books:

	user_id	book_id
0	A3HHXRELK8BHQG	B0033UV8HI
1	A2RGNZ0TRF578I	B002HJV4DE
2	A3S0H2HV6U1I7F	B002ZG96I4
3	AC40QW3GZ919J	B002QHW0EU
4	A3C9V987IQHOQD	B001A06VJ8
...	...	...
77141	Carol McCoy	1492646318
77142	Chelscey	1501161938
77143	bayrider	385534264
77144	Amazon Customer	B0B6XFT4RH
77145	Jeanette F Reeder	1728213746

[77126 rows x 2 columns]

```
In [9]: 1 valid_user_ids = set(cl_user_df['user_id'])
2         valid_book_ids = set(cl_book_df['book_id'])
3
4         # Ratings Checks
5         invalid_ratings_user = cl_rating_df[~cl_rating_df['user_id'].isin(valid_user_ids)]
6         invalid_ratings_book = cl_rating_df[~cl_rating_df['book_id'].isin(valid_book_ids)]
7
8         print(" Foreign Key Checks for Ratings ")
9         print(f"Invalid User IDs: {len(invalid_ratings_user)}")
10        print(f"Invalid Book IDs: {len(invalid_ratings_book)}")
11
12
13        # Cleaned Review Checks
14        invalid_review_user = cl_pre_df[~cl_pre_df['user_id'].isin(valid_user_ids)]
15        invalid_review_book = cl_pre_df[~cl_pre_df['book_id'].isin(valid_book_ids)]
16
17        print(" \nForeign Key Checks for Reviews ")
18        print(f"Invalid User IDs: {len(invalid_review_user)}")
19        print(f"Invalid Book IDs: {len(invalid_review_book)}")
```

Foreign Key Checks for Ratings
Invalid User IDs: 2177
Invalid Book IDs: 49829

Foreign Key Checks for Reviews
Invalid User IDs: 77146
Invalid Book IDs: 77126

iii. Check for Missing Values Post-Cleaning

- Count null values in all cleaned datasets to confirm no accidental loss.

```
In [10]: 1 def check_nulls(df, name):
2         print(f"\n{name} - Null Value Check:")
3         print(df.isnull().sum())
4
5         check_nulls(cl_book_df, "Books_Cleaned")
6         check_nulls(cl_user_df, "Users_Cleaned")
7         check_nulls(cl_rating_df, "Ratings_Cleaned")
8         check_nulls(cl_merge_review_df, "Merge_Review_Cleaned")
9         check_nulls(cl_pre_df, "Preprocessed_Review")
10
```

Books_Cleaned - Null Value Check:

```
book_id      0
title        0
author       0
Year-Of-Publication  0
publisher    0
Image-URL-S  0
Image-URL-M  0
Image-URL-L  0
dtype: int64
```

Users_Cleaned - Null Value Check:

```
user_id      0
location     0
age          0
dtype: int64
```

Ratings_Cleaned - Null Value Check:

```
user_id      0
book_id      0
ratings      0
dtype: int64
```

Merge_Review_Cleaned - Null Value Check:

```
review_id    0
book_id      0
user_id      0
review_text   5
ratings      0
review_date   0
source       0
author       0
book_name    0
dtype: int64
```

Preprocessed_Review - Null Value Check:

```
review_id    0
book_id      0
user_id      0
review_text   0
ratings      0
review_date   0
source       0
author       0
book_name    0
cleaned_tokens 0
dtype: int64
```

iv. Row Count Before vs After Cleaning

- Compared original and cleaned row counts to detect any accidental deletions.
- Save and compare row counts before and after cleaning.

```
In [11]: 1 print("ROW COUNT COMPARISON (before -> after)")
2 print(f"Books: {len(book_df)} → {len(cl_book_df)}")
3 print(f"Users: {len(user_df)} → {len(cl_user_df)}")
4 print(f"Ratings: {len(rating_df)} → {len(cl_rating_df)}")
5 print(f"Reviews (Merged): {len(cl_merge_review_df)}")
6 print(f"Reviews (Preprocessed): {len(cl_pre_df)}")
```

```
ROW COUNT COMPARISON (before -> after)
Books: 271360 → 271360
Users: 278858 → 277610
Ratings: 1149780 → 433671
Reviews (Merged): 77146
Reviews (Preprocessed): 77146
```

v. Null Value Checks

- Count null values in all cleaned datasets to confirm no accidental loss.

```
In [12]: 1 def check_nulls(df, name):
2         print(f"\n{name} Null Value Summary:\n")
3         print(df.isnull().sum())
4
5         check_nulls(cl_book_df, "Books_Cleaned")
6         check_nulls(cl_user_df, "Users_Cleaned")
7         check_nulls(cl_rating_df, "Ratings_Cleaned")
8         check_nulls(cl_merge_review_df, "Merge_Review_Cleaned")
9         check_nulls(cl_pre_df, "Preprocessed_Review")
10
```

Books_Cleaned Null Value Summary:

```
book_id      0
title        0
author       0
Year-Of-Publication  0
publisher    0
Image-URL-S  0
Image-URL-M  0
Image-URL-L  0
dtype: int64
```

Users_Cleaned Null Value Summary:

```
user_id      0
location     0
age          0
dtype: int64
```

Ratings_Cleaned Null Value Summary:

```
user_id      0
book_id      0
ratings      0
dtype: int64
```

Merge_Review_Cleaned Null Value Summary:

```
review_id    0
book_id      0
user_id      0
review_text   5
ratings      0
review_date  0
source       0
author       0
book_name    0
dtype: int64
```

Preprocessed_Review Null Value Summary:

```
review_id    0
book_id      0
user_id      0
review_text   0
ratings      0
review_date  0
source       0
author       0
book_name    0
cleaned_tokens  0
dtype: int64
```

vi. Sample Data Inspection

```
In [13]: 1 # Function to display sample rows
2 def show_sample(df, name, n=3):
3     print(f"\n{name} Sample ({n} rows)")
4     display(df.sample(n))
5
6 # View samples from each cleaned dataset
7 show_sample(cl_book_df, "Books_Cleaned")
8 show_sample(cl_user_df, "Users_Cleaned")
9 show_sample(cl_rating_df, "Ratings_Cleaned")
10 show_sample(cl_merge_review_df, "Merge_Review_Cleaned")
11 show_sample(cl_pre_df, "Preprocessed_Review")
12
```

Books_Cleaned Sample (3 rows)

	book_id	title	author	Year-Of-Publication	publisher	Image-URL-S	Image-URL-M
163725	0670837660	Goodbye Jumbo: Hello Cruel World	Louie Anderson	1993	Penguin USA	http://images.amazon.com/images/P/0670837660.0...	http://images.amazon.com/images/P/0670837660.0...
22986	0553269291	Rubyfruit Jungle	Rita Mae Brown	1983	Bantam Books	http://images.amazon.com/images/P/0553269291.0...	http://images.amazon.com/images/P/0553269291.0...
77254	1553690257	An Ia Story: One Thread in the Tapestry of Con...	S. A. Stanley	2002	Not Avail	http://images.amazon.com/images/P/1553690257.0...	http://images.amazon.com/images/P/1553690257.0...

Users_Cleaned Sample (3 rows)

	user_id	location	age
210049	210984	toronto, ontario, canada	32.0
213069	214025	dartmouth, nova scotia, canada	32.0
41921	42119	san diego, california, usa	32.0

Ratings_Cleaned Sample (3 rows)

	user_id	book_id	ratings
160379	97874	0525947655	10
147814	90320	0671734563	10
209255	126492	0399150838	6

Merge_Review_Cleaned Sample (3 rows)

	review_id	book_id	user_id	review_text	ratings	review_date	source	author	book_name
20516	20517	unknown	unknown	As I sit here digesting the monumental feast o...	4	2025-01-01	Goodreads	Unknown	Unknown
46904	46905	unknown	unknown	Update I found an old review I wrote about thi...	1	2025-01-01	Goodreads	Unknown	Unknown
3168	3169	B0038KX8S0	A1CF279EEYJVUC	Normally Im not picky about editing or grammar...	3	2013-10-20	Amazon	Unknown	Unknown

Preprocessed_Review Sample (3 rows)

	review_id	book_id	user_id	review_text	ratings	review_date	source	author	book_name	cleaned_tokens
24129	24130	unknown	unknown	It is soo important what comes out of your mou...	4	2025-01-01	Goodreads	Unknown	Unknown	soo important comes mouth want gods blessings
63650	63651	unknown	unknown	Id file this under historical fiction light no...	4	2025-01-01	Goodreads	Unknown	Unknown	id file historical fiction light unlike boley...
33147	33148	unknown	unknown	Really enjoyed this one and compared to other ...	5	2025-01-01	Goodreads	Unknown	Unknown	really enjoyed one compared time management bo...

vii. Duplicate Checks

- Checked for duplicated values to ensure integrity.

```
In [14]: 1 print("Duplicate Check")
2 print(f"Books: {cl_book_df.duplicated().sum()} duplicates")
3 print(f"Users: {cl_user_df.duplicated().sum()} duplicates")
4 print(f"Ratings: {cl_rating_df.duplicated().sum()} duplicates")
5 print(f"Merged Reviews: {cl_merge_review_df.duplicated().sum()} duplicates")
6 print(f"Preprocessed Reviews: {cl_pre_df.duplicated().sum()} duplicates")

Duplicate Check
Books: 0 duplicates
Users: 0 duplicates
Ratings: 0 duplicates
Merged Reviews: 0 duplicates
Preprocessed Reviews: 0 duplicates
```

4. Sprint 1 Review & Retrospective

i. What Was Accomplished

- Loaded and initially cleaned all datasets (books, users, ratings, reviews).
- Resolved encoding issues and filtered out corrupted/unreadable text.
- Performed validation checks:
 - a. Handled null values
 - b. Checked and corrected data types
 - c. Removed duplicate entries.
- Summary stats and basic structure analysis completed.
- Review text was preprocessed for future NLP use.
- Resolved formatting and column name mismatches across datasets.

ii. Challenges Faced

- Encoding issues in review text (handled with ASCII filter)
- Inconsistent structures and column naming in some datasets.
- Detected and removed duplicate entries in the ratings dataset.
- Whitespace or special characters in column names caused merge/join errors.

iii. Improvements for Next Sprint

- Track changes with better version control (e.g., copies before/after cleaning)
- Finalize foreign key integrity checks.
- Setup integrated validation pipeline for nulls, consistency, and types.
- Automate the data validation using test scripts.

- Add visual dashboards for data overview.
- Introduce unit tests for key data pipelines.